# dSABRE: A SABRE-Style Router for Multi-Core Distributed Quantum Computers

Sanjiang Li *

***Abstract*—Minimising EPR consumption is the dominant objective when routing a quantum circuit on a distributed quantum computer (DQC). We present DSABRE, a SABRE-style router for multi-core processors that, on each iteration of a lookahead-driven loop, first resolves any intra-core front-layer gates by SWAP scoring and only falls back to scoring inter-core teleportation candidates when the intra-core front is empty. Three mechanisms drive the improvement over the state of the art: a five-term *gate-centric* teleportation score that generalises the local SWAP heuristic to the inter-core setting, whose explicit capacity-penalty term keeps the scorer from teleporting into saturated cores; a proactive congestion-relief pass that redistributes idle qubits out of high-demand cores before deadlock; and a BFS-layer construction of the inter-core extended set that respects DAG dependencies layer by layer rather than mixing wires in topological order. Across 18 MQT-Bench circuits at 25, 36, and 64 logical qubits, DSABRE reduces geometric-mean EPR consumption by 41–53% over TELESABRE. Against the gate-teleportation-based `pytket-dqc` using its strongest Steiner-embedding distributor, DSABRE reduces EPRs by 12–57% on the 25- and 36-qubit suites; on the denser 64-qubit suite that method's gate embedding is more e-bit-efficient, though it neither accounts for intra-core SWAP cost nor scales to the largest circuits. DSABRE uses a `SabreLayout`-based initial layout with per-core communication-slot reservation. A large-circuit QFT sweep at 100–360 qubits confirms scalability. Code and online appendices are available at https://github.com/ebony72/dsabre.**

***Index Terms*—distributed quantum computing, circuit routing, qubit teleportation, SABRE, EPR minimisation, qubit mapping, lookahead heuristic**

## I. INTRODUCTION

The near-term path to fault-tolerant quantum computing requires processors with thousands of high-fidelity qubits. No single monolithic chip can economically deliver this scale today. Instead, the field is converging on *distributed quantum computers* (DQC) [1]: multiple small quantum processing units (QPUs, also called cores) interconnected by photonic or microwave links that support remote entanglement generation.

Executing a quantum circuit on a DQC requires a *distributed compilation* pipeline that, like its monolithic counterpart, can be decomposed into three problem classes [2]. *(i) Qubit allocation* statically assigns logical qubits to cores — e.g. via hypergraph partitioning [3], [4]. *(ii) Communication-primitive selection* decides how each non-local 2Q gate is realised: *state teleportation* moves a logical qubit across an inter-core link [5]; *gate teleportation* (telegate / EJPP / cat-entanglers) executes a single non-local gate without moving the qubit [6], [7], optionally amortising a sequence of consecutive non-local gates onto one shared e-bit [8]. Each primitive consumes one or more EPR pairs, and EPR generation is slow (microseconds to milliseconds), noisy, and rate-limited in current hardware [9], [10], making EPR minimisation the dominant compilation objective. *(iii) Routing and scheduling* orders intra-core SWAPs, teleports, and EPR-generation requests in time, subject to communication-port capacity and finite e-bit rates [11], [12].

DSABRE addresses the routing component of stage (iii) — ordering intra-core SWAPs and teleports — and leaves scheduling (EPR-generation timing, port reservation) to downstream passes; within stage (ii) it uses only state teleportation, and it is complementary to the static-allocation methods of stage (i). Within this scope the closest prior work is the family of SABRE-style [13] distributed routers, which score teleportation candidates with a heuristic that combines an immediate front-layer distance reduction and a lookahead term over a fixed window of upcoming gates [14]. The most recent and best-performing of these is TELESABRE [15], which uses a Dijkstra computation on a contracted communication graph to estimate inter-core routing cost. We observe one substantive limitation: congestion is handled defensively. TELESABRE discourages landing in saturated cores via a full-core penalty and recovers from deadlock via a safety-valve mode that drops most of the lookahead, but it never proactively *evicts* idle qubits out of crowded cores ahead of demand. On dense or large instances this recovery either burns additional EPR pairs or fails to converge altogether.

**Contributions.**

- **A five-term gate-centric teleportation score.** LIGHTSABRE [16] scores a SWAP as the front-layer distance gain $\Delta_F$ plus a decay-weighted lookahead $\Delta_E$, both in $\Delta$-form (old minus new distance) on gates sharing a qubit with the move. DSABRE carries the same two $\Delta$-form terms into the inter-core scorer and adds three teleport-specific terms: a staging SWAP cost $d_{\text{prep}}$ to reach a comm port, a *core-capacity penalty* $c_{\text{cap}}$ that discourages landing in nearly-full cores, and a hop-direction reward $g_{\text{hop}}$. Ablation (Section IV-C) shows the capacity term carries the score: removing it inflates geometric-mean EPR by $+60.8\%$ on the 25-qubit suite and $+53.5\%$ on the 64-qubit suite, more than any other single mechanism; removing the lookahead adds another $+11.3\%$ on the 25-qubit suite.
- **Proactive congestion relief.** DSABRE maintains an explicit demand vector over cores and proactively teleports idle qubits away from high-demand, low-capacity cores

S. Li is with the Centre for Quantum Software and Information, University of Technology Sydney, Australia (e-mail: sanjiang.li@uts.edu.au).
*ORCID: 0000-0002-3332-2546.

before bottlenecks form, scoring relief moves within the same teleportation objective. On the per-core adaptive layout its geometric-mean effect is small—disabling relief *lowers* 64-qubit gmean EPR by 8.7% (Section IV-C)—indicating that much of the congestion it once addressed was an artifact of the previous core-0-only layout that born several cores fully packed. Its remaining value is on individual hard instances: on the 64-qubit Random circuit, DSABRE completes routing under proactive relief where TELESABRE fails to converge.

- **BFS-layer inter-core extended set.** The inter-core scorer builds its lookahead set layer by layer over the remaining DAG so that gates sharing a wire with the front are captured before unrelated wires displace them, with each gate's BFS depth driving the decay exponent $\gamma^{\mathrm{dep}(g)}$. Substituting a topological-order extended set increases geometric-mean EPR by 3–11% across the three suites, with the gain growing with circuit size (Section IV-C).
- **Comprehensive empirical validation.** Across MQT-Bench suites at 25, 36, and 64 qubits on multi-core grid architectures, DSABRE reduces geometric-mean EPR consumption by 41–53% over TELESABRE [15]. It also reduces EPRs by 12–57% over `pytket-dqc` [4] (strongest Steiner-embedding distributor) on the 25- and 36-qubit suites, while on the 64-qubit suite that method's gate embedding is more e-bit-efficient on structured circuits—at the cost of ignoring intra-core SWAPs and of runtimes orders of magnitude longer. DSABRE uses a `SabreLayout`-based initial layout with per-core communication-slot reservation. Ablation studies, cost-ratio sweeps, and a scalability sweep up to 360-qubit QFT confirm that the gains are mainly driven by the heuristic design and congestion mechanisms.

**Paper organisation.** Section II fixes notation and recaps the SABRE heuristic. Section III presents DSABRE: the routing loop, the intra-core and inter-core scoring rules, the BFS-layer extended set, proactive congestion relief, the checkpoint–rollback escape, and a complexity analysis. Section IV evaluates DSABRE on the 25q, 36q, and 64q MQT-Bench suites against TELESABRE and `pytket-dqc`, ablates the design choices, characterises the initial-layout pipeline, reports compile time, and runs a 100–360q QFT scalability sweep. Section V positions DSABRE against related work; Section VI outlines directions for further work; Section VII concludes.

## II. Background

### A. Quantum Circuits

A quantum circuit acts on $n$ *qubits* through a sequence of *gates*. *One-qubit (1Q) gates* (Hadamard, rotations, …) act on a single qubit. *Two-qubit (2Q) gates* entangle pairs: the *CX* (CNOT) gate is the standard primitive and the building block for most multi-qubit operations. Together, CX and arbitrary 1Q gates form a universal gate set, so any $n$-qubit unitary can be compiled into a circuit over this basis. The *SWAP* gate exchanges the states of two adjacent qubits; it decomposes into three CX gates and is used by routing algorithms to migrate logical qubit states along the coupling graph.

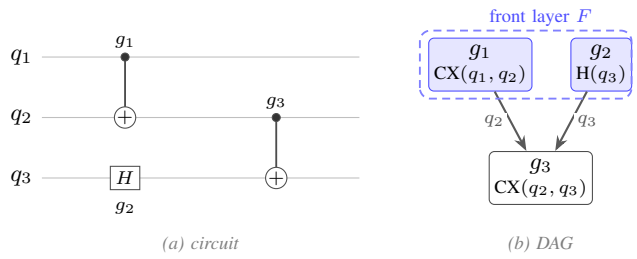



Fig. 1. DAG for a three-gate circuit on qubits $q_1, q_2, q_3$. $\mathrm{CX}(q_1, q_2)$ and $\mathrm{H}(q_3)$ share no qubit and have no incoming edges, so both appear in the initial front layer $F = \{g_1, g_2\}$ and may execute in parallel. $\mathrm{CX}(q_2, q_3)$ depends on both via qubits $q_2$ and $q_3$; it enters $F$ only after $g_1$ and $g_2$ have been executed. Circuit depth is 2 (two sequential layers).

A circuit is represented as a *directed acyclic graph* (DAG) in which each node is a gate and each directed edge records a data dependency (qubit produced by gate $g_1$ consumed by gate $g_2$). The construction rule is: for every qubit $q$, connect each gate acting on $q$ to the *next* gate acting on $q$ by a directed edge; the edge means the second gate cannot begin until the first has written its output on $q$. One-qubit gates therefore contribute at most one outgoing edge per operand, while 2Q gates contribute one per operand pair. Figure 1 shows a small example, where *circuit depth* is the length of the longest path through the DAG.

Physical qubits are arranged in a *coupling graph* $G_{\mathrm{phys}} = (P, E_{\mathrm{phys}})$; a 2Q gate can execute only if its two operands occupy adjacent vertices. A circuit is written in terms of *logical qubits* $\mathrm{q}_1, \ldots, \mathrm{q}_n$; a *layout* $\ell : \mathrm{q}_i \mapsto \mathrm{p}_p$ assigns each to a physical slot. *Qubit routing* inserts SWAP gates so that the two operands of every 2Q gate reach adjacent physical qubits before it executes.

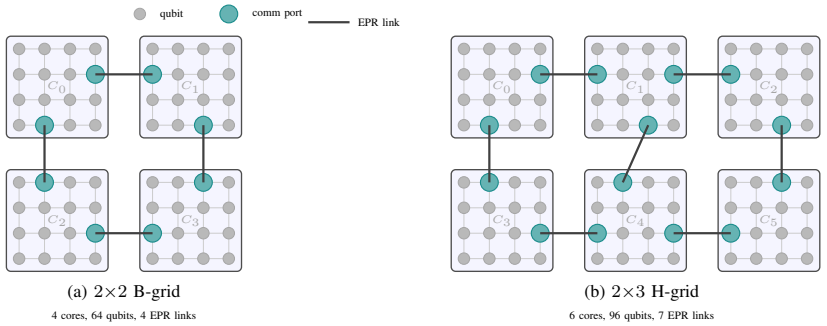
### B. Quantum Teleportation

*Quantum teleportation* [5] transfers the state of a qubit from a sender to a receiver—in the DQC setting, from one core to another over an inter-core link—using a pre-shared *EPR pair* (a maximally entangled two-qubit state) and two classical bits. The source qubit's state is destroyed at the sender and reconstructed at the receiver; no physical qubit travels across the link. Each EPR pair is consumed upon use, so the *EPR count* is the primary inter-core routing cost in distributed quantum compilation. Throughout this paper, "EPR" as a unit (e.g. "10 EPRs") denotes EPR-pair count; "EPR pair" is used when emphasising the physical entangled state itself.

Two inter-core teleportation variants arise in DQC compilation. *Teledata* moves a logical qubit to a new core (one EPR pair consumed), after which the qubit participates in local gates there. *Telegate* [6] executes a non-local 2Q gate directly across two cores via a shared EPR pair, without physically relocating either qubit. DSABRE uses teledata exclusively; TELESABRE exploits both variants.

### C. Distributed Quantum Architecture

We model a DQC architecture $\mathcal{A}$ abstractly as a weighted graph $G_{\mathcal{A}} = (V, E, w)$ whose vertices are physical qubits. The vertex set is partitioned into $K$ *cores* $V = V_1 \sqcup \cdots \sqcup V_K$;




(a) 2×2 B-grid
4 cores, 64 qubits, 4 EPR links

(b) 2×3 H-grid
6 cores, 96 qubits, 7 EPR links

Fig. 2. DQC architectures used in this paper (B-grid and H-grid families introduced by TELESABRE [15]). Small grey circles are qubits; teal circles are inter-core communication ports; bold lines are EPR-channel links between cores. Each core is a 4×4 grid. (a) 2×2 B-grid. (b) 2×3 H-grid.

intra-core edges $E_{\text{intra}}$ describe in-core connectivity and carry weight 1, and inter-core edges $E_{\text{inter}}$ connect designated *communication ports* on neighbouring cores and carry a much larger weight $w_{\text{link}} \gg 1$ reflecting the cost of an EPR pair. A communication port is a physical qubit like any other and may also hold a logical (data) qubit between teleportations; the "communication" label only indicates that the port is one end of an inter-core EPR link, not that the slot is reserved. DSABRE treats $G_{\mathcal{A}}$ as a black box and only queries it through three distance tables: physical $d_{\text{phys}}$, intra-core $d_{\text{intra}}$, and core-graph $d_{\text{core}}$. The router therefore extends to *any* DQC topology — arbitrary intra-core graphs, mixed core sizes, sparse inter-core meshes, hierarchical trees of clusters, etc. — by simply supplying these tables.

For experimental simplicity our reference implementation instantiates $G_{\mathcal{A}}$ as a *grid-of-grids* [15]: each core is an $m \times m$ 2D nearest-neighbour grid, and the cores themselves form an $r \times s$ super-grid joined by a small set of inter-core links between specific boundary qubits. Figure 2 shows two concrete topologies used in our experiments.

*a) Hierarchical distance precomputation.:* Because intra-core and inter-core costs decouple, the all-pairs distance $d_{\text{phys}}$ over $G_{\mathcal{A}}$ admits a two-level Dijkstra computation that is substantially cheaper than running Dijkstra on the full $|V|$-vertex graph. First, all-pairs distances are computed once *inside* a single core $V_i$ (size $|V_i| = M$) in $O(M^2 \log M)$ time, giving $d_{\text{intra}}$. Second, the *core graph* (a multigraph of $K$ super-nodes connected by inter-core links) is solved in $O(K^2 \log K)$ time, giving $d_{\text{core}}$. Third, $d_{\text{phys}}(p, q)$ for any $p \in V_i$, $q \in V_j$ is recovered as $d_{\text{intra}}(p, \pi_i^*) + d_{\text{core}}(i, j) + d_{\text{intra}}(\pi_j^*, q)$ minimised over boundary-port pairs $(\pi_i^*, \pi_j^*)$ realising the optimal core-graph path. This brings precomputation from $O((KM)^2 \log(KM))$ down to $O(KM^2 \log M + K^2 \log K)$ and is the only architecture-dependent setup DSABRE performs; routing itself uses $O(1)$ table lookups thereafter.

### D. Qubit Allocation and Routing

Given circuit DAG $C$ and initial layout $\ell : \mathrm{q}_i \mapsto \mathrm{p}_{\sigma(i)}$ ($\sigma$ is a permutation on $0, 1, ..., n-1$ with $n$ the number of physical qubits), routing inserts the minimum number of SWAP and teleportation operations so every gate can execute on adjacent physical qubits. Cost model: each local SWAP costs $c_{\text{swap}}$; each inter-core teleport costs $c_{\text{tele}}$ (one EPR pair).

### E. SABRE

LIGHTSABRE [16] optimises SABRE [13] by selecting the SWAP minimising:

$$H = \frac{1}{|F|} \sum_{g \in F} \Delta_F(g) + w \frac{1}{|E|} \sum_{g \in E} \Delta_E(g), \tag{1}$$

where $F$ is the front layer, $E$ the extended lookahead set, and $\Delta(g) = d^g_{\text{old}} - d^g_{\text{new}}$ the reduction in physical distance between the two qubits of gate $g$ caused by the candidate SWAP ($d^g$ is shortest-path distance on the coupling graph). All extended-set gates contribute with equal weight; the $\frac{1}{|E|}$ factor normalises the lookahead term to the same scale as the front term. SABRE also couples its router with an initial-layout optimiser: starting from a random qubit assignment it routes the circuit $C$ forward, then immediately routes the reverse circuit $C^{-1}$ using the final layout of the forward pass as the new starting point; the layout produced by the reverse pass is fed back as the initial layout for the next forward pass. After several bidirectional sweeps the heuristic has annealed towards a layout that is self-consistent with the circuit's gate structure.

## III. DSABRE

DSABRE is a SABRE-style routing algorithm for multi-core processors. At the intra-core level its SWAP-selection objective inherits the front-layer / extended-set scoring of Eq. 1, with two refinements: a decay-weighted lookahead ($\gamma^{\text{dep}(g)}$ down-weights deep successors) and taint propagation that excludes qubits already entangled with cross-core gates (Section III-B). Inter-core movement is handled by a separate teleportation scorer (Section III-C).

Routing assumes a fixed initial layout; we use Qiskit `SabreLayout`, and defer the discussion of layout preparation and the routing pass schedule to Section IV-D.

The core invariants and notation are: (i) the layout $\ell : \mathrm{q}_q \to \mathrm{p}_p$ assigns logical to physical qubits and the inverse $\ell^{-1}$ records which logical qubit (if any) currently occupies each physical slot; (ii) the working DAG $W$ holds the remaining gates (initially the full circuit) and shrinks as gates are executed; (iii) the front layer $F$ is the set of gates in $W$ with no unexecuted predecessors, and the extended set $E$ is a fixed-size sliding lookahead beyond $F$.

**Algorithm 1** DSABRE `route`$(C, \ell_0)$

$W \leftarrow$ working copy of circuit DAG $C$; $\ell \leftarrow \ell_0$
$C_{\text{out}} \leftarrow$ empty routed circuit on the physical qubits
save checkpoint $(\ell, W)$
**while** $W$ has unexecuted gates **do**
  drain $F$ (execute 1Q gates and adjacent intra-core 2Q gates; append them to $C_{\text{out}}$) *(P1)*
  **if** $W$ empty **then**
    **break**
  **end if**
  compute $F$, partition into $F_{\text{intra}}, F_{\text{inter}}$ *(P2)*
  **if** $F_{\text{intra}} \neq \emptyset$ **then**
    compute per-core intra-only extended set $E_c$ for each active core $c$ *(P3a)*
    pick best SWAP $(u, v)$ within a core (Eq. 3)
    apply SWAP$(u, v)$ to $\ell$; append SWAP$(u, v)$ to $C_{\text{out}}$
  **else if** $F_{\text{inter}} \neq \emptyset$ **then**
    compute global $E$ if not cached *(P3b)*
    generate candidates over $F_{\text{inter}}$ and relief moves
    pick lowest-score candidate (Eq. 4)
    apply teleport to $\ell$; append teleport (staging SWAPs + teledata) to $C_{\text{out}}$
  **end if**
  **if** $|W|$ decreased **then**
    save checkpoint *(P4)*
  **else if** $L_{\text{deadlock}}$=50 stalled iterations **then**
    restore checkpoint; `backup_plan`$(W, \ell, C_{\text{out}})$ // *abort after* $N^{\max}_{\text{backup}}$=50
  **end if**
**end while**
**return** $(\ell, C_{\text{out}}, \text{metrics})$

### A. Workflow

Algorithm 1 and Figure 3 sketch the main routing loop. Each iteration runs four phases: **(P1)** drain $F$ (execute 1Q gates and any 2Q gate whose operands are already adjacent in the same core, iterated to a fixed point); **(P2)** partition the remaining front into $F_{\text{intra}}$ (both operands in one core) and $F_{\text{inter}}$ (operands in different cores); **(P3)** if $F_{\text{intra}} \neq \emptyset$ apply one intra-core SWAP (Section III-B), else score teleportation candidates over $F_{\text{inter}}$ together with proactive relief candidates and apply the cheapest (Section III-C); **(P4)** checkpoint $(\ell, W)$ if $|W|$ decreased, otherwise trigger checkpoint–rollback after $L_{\text{deadlock}}$ stalled iterations and abort after $N^{\max}_{\text{backup}}$ failed recoveries. The two scoring functions operate at different scopes: the intra-core scorer is per-core, restricted to intra-core continuations (qubits involved in inter-core gates are tainted), and front-/extended-set averaged in the SABRE style; the inter-core scorer evaluates moves across cores using the global $F_{\text{inter}}$ and a global BFS-layer extended set (Section III-D), with $\tilde{\Delta}_E$ entering as an un-averaged decay-weighted sum because the cross-core lookahead window is small and its size already enters implicitly via the $|E|_{\max}$ cap.

### B. Intra-Core SWAP Heuristic

When the front contains a gate whose operands share a core but are not adjacent, DSABRE chooses an intra-core SWAP using the standard SABRE heuristic, restricted to the relevant core. Let $F_c = \{g \in F_{\text{intra}} : \text{core}(g) = c\}$ be the front gates in core $c$ and $E_c = (g_0, g_1, \ldots)$ the corresponding intra-core extended set, an *ordered list* of at most $L$ gates (default $L$=20) collected in topological order. The traversal stops including a gate the moment either of its qubits has previously appeared in a cross-core gate in the working DAG (taint propagation), ensuring $E_c$ contains only gates that remain local to core $c$. Each gate $g_i$ is assigned a depth $\text{dep}(g_i)$, the number of intra-core gates on the longest qubit-wire path from the front layer to $g_i$ (front layer = depth 0; first successor = depth 1). The lookahead contribution is

$$\tilde{\Delta}^{c}_{E} = \sum_{i=0}^{|E_c|-1} \gamma^{\text{dep}(g_i)} (d^{g_i}_{\text{old}} - d^{g_i}_{\text{new}}), \tag{2}$$

where $\gamma \in (0, 1]$ is the lookahead decay and the distance terms use the intra-core graph. The tilde marks the departure from SABRE's flat extended-set weighting: rather than treating all lookahead gates equally, the exponential factor $\gamma^{\text{dep}(g_i)}$ down-weights gates far from the front so that near-term successors dominate the lookahead signal — a deeper gate is more likely to be displaced by intervening routing decisions and so contributes less reliable information about the right SWAP now. The same weighted scheme is reused by the inter-core scorer (Section III-C); Section III-D explains how $\gamma^{\text{dep}(g)}$ interacts with the BFS-layer extended set, where $\text{dep}(g)$ is the BFS depth rather than an iteration index. For every neighbour pair $(u, v)$ within a core, the score

$$H_{\text{intra}} = \frac{\Delta^{c}_{F}}{|F_c|} + w_e \frac{\tilde{\Delta}^{c}_{E}}{|E_c|} \tag{3}$$

is the SABRE objective for intra-core scoring with the lookahead-decay weighting of Eq. 2. The lowest-score SWAP is applied; tie-breaking is by enumeration order. Because the scorer is restricted to a single core and the candidate set is the $O(M)$ intra-core neighbour pairs, the per-core selection is trivially parallel across active cores.

### C. Inter-Core Teleportation Scoring

For each gate $g \in F_{\text{inter}}$ on logical qubits $q_1, q_2$, let $c_{\text{src}}$ and $c_{\text{tgt}}$ be the cores currently hosting $q_1$ and $q_2$ respectively (i.e. $\ell(q_1) \in c_{\text{src}}$ and $\ell(q_2) \in c_{\text{tgt}}$, where $\ell$ is the current layout). Both endpoints are tried as teleportation sources. For the qubit chosen as source (say $q_1$ at physical position $p_1 = \ell(q_1)$), the router enumerates every neighbouring core $c_{\text{next}}$ and every inter-core link $(\pi_s, \pi_d)$ between $c_{\text{src}}$ and $c_{\text{next}}$. Each such triple $(q_1, c_{\text{next}}, (\pi_s, \pi_d))$ is one *candidate*; Figure 4 illustrates the structure. Executing a candidate involves two steps: (1) intra-core SWAPs that move $q_1$ from $p_1$ to the *staging slot* $n_s$, the neighbour of $\pi_s$ closest to $p_1$; and (2) a teleportation that consumes the EPR pair on link $(\pi_s, \pi_d)$ and delivers $q_1$ from $n_s$ to $\pi_d$ in $c_{\text{next}}$. Each candidate is scored as:

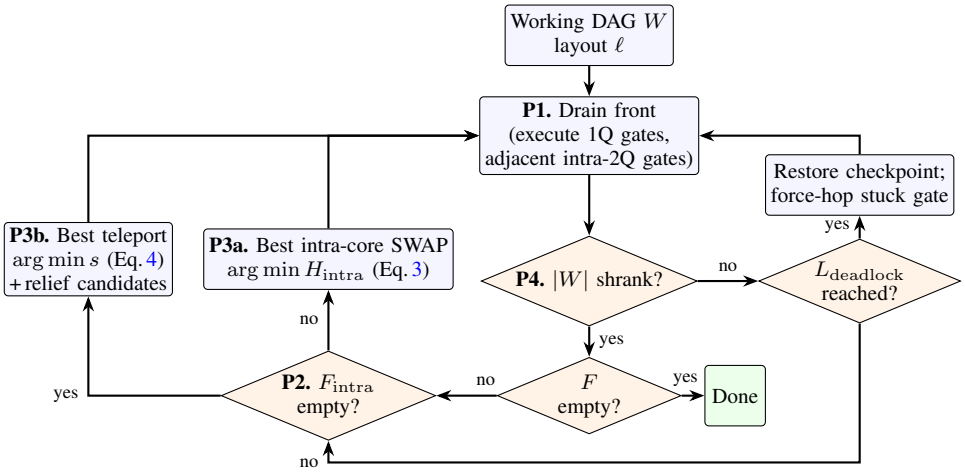



Fig. 3. The DSABRE routing workflow. Each iteration drains the front layer (P1), classifies the remaining front into intra-core and inter-core gates (P2), applies one SWAP or one teleport (P3), and checkpoints when forward progress is made (P4).

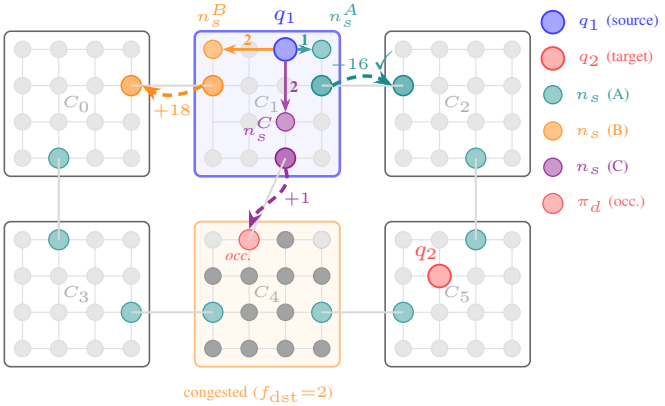



Fig. 4. Running example for inter-core teleportation scoring (2×3 H-grid). Candidates **A**, **B**, and **C** below correspond row-by-row to Table I. Gate $g = (q_1, q_2)$ has $q_1$ in source core $C_1$ (blue border) and $q_2$ in target core $C_5$. The router considers three candidates, each a triple $(q_1, c_{\text{next}}, (\pi_s, \pi_d))$: **A** (teal) routes $q_1$ towards $C_2$; **B** (orange) routes towards $C_0$, away from $C_5$; **C** (violet) routes towards congested landing core $C_4$ (orange border, $f_{\text{dst}}$=2, where $f_{\text{dst}}$ counts free slots in $c_{\text{next}}$, not in $c_{\text{tgt}}$). Step (1): intra-core SWAPs move $q_1$ from $p_1$ to staging slot $n_s$ (count shown on arrow). Step (2): a teleportation along link $(\pi_s, \pi_d)$ delivers $q_1$ to $\pi_d$ in $c_{\text{next}}$ (dashed arc; score $s$ annotated). Candidate A wins ($s_A$=−16) by combining the fewest staging SWAPs with positive $\Delta_F$ and hop gain and no capacity penalty. The red port ($\pi_d^C$, labelled *occ.*) indicates an occupied landing port, contributing an extra eviction SWAP to $d_{\text{prep}}^C$.

$$s = \underbrace{d_{\text{prep}}}_{\text{staging}} + \underbrace{c_{\text{cap}}}_{\text{capacity}} - \underbrace{g_{\text{hop}}}_{\text{hop gain}} - \underbrace{\Delta_F}_{\text{front gain}} - \underbrace{w_e \tilde{\Delta}_E}_{\text{lookahead}}, \qquad (4)$$

where lower scores are preferred. Each term is defined as follows.

**Staging cost** $d_{\text{prep}}$ is the number of intra-core SWAPs needed to move $q_1$ from $p_1$ to staging slot $n_s$, plus eviction SWAPs to free the comm ports if occupied:

$$d_{\text{prep}} = d_{\text{intra}}(p_1,\, n_s) + d_{\text{evict}}(\pi_s) + d_{\text{evict}}(\pi_d).$$

**Capacity penalty** $c_{\text{cap}} = c_{\text{pen}}\ \max(0,\, \tau - f_{\text{dst}})$ discourages routing into nearly-full cores. $f_{\text{dst}}$ is the number of free slots in the immediate landing core $c_{\text{next}}$ (not the final partner-qubit core $c_{\text{tgt}}$), $\tau$ the capacity threshold, and $c_{\text{pen}}$ a cost multiplier. This prevents progressive crowding of hot cores that leads to deadlock.

**Front-layer gain** $\Delta_F = d_{\text{old}}^g - d_{\text{new}}^g$ is the reduction in weighted physical distance between $q_1$ and $q_2$ after the teleport, computed on the full-chip graph $G_{\mathcal{A}}$ (intra-core edges weight 1, inter-core links weight $w_{\text{link}}$). There is exactly one front-layer gate involving the moving qubit $q_1$: because any two gates that share a qubit are ordered by a dependency edge in the DAG, at most one of them can be unblocked (i.e. in $F$) at any time, and $g = (q_1, q_2) \in F_{\text{inter}}$ is already that gate by the premise of this scoring step.

**Hop gain** $g_{\text{hop}} = w_h\,(d_{\text{core}}(c_{\text{src}}, c_{\text{tgt}}) - d_{\text{core}}(c_{\text{next}}, c_{\text{tgt}}))$ is a position-independent directional reward that supplements $\Delta_F$ when the landing port $\pi_d$ is far from the best staging position for the next hop, causing $\Delta_F$ to under-report core-level progress. The term is largely redundant with $\Delta_F$ on small topologies (the B-grid suites tie exactly with $g_{\text{hop}}$ ablated) and contributes only on larger H-grids (+1.8% geometric-mean EPR on the 64-qubit suite, Section IV-C); its importance is expected to grow on architectures with larger cores, where the landing port can sit many SWAPs away from the next staging position.

**Decay-weighted lookahead.** $\tilde{\Delta}_E$ is the same decay-weighted lookahead introduced in Section III-B (Eq. 2), reused here with two adaptations for the inter-core scorer: $E$ is now the inter-core extended set (Section III-D) rather than the per-core taint-propagated list, and the sum is restricted to gates involving the teleported qubit, since only those gates are affected by the candidate move. The same $\gamma$ controls how rapidly deeper successors are discounted.

*a) Running example.:* Figure 4 illustrates Eq. 4 on the 2×3 H-grid with defaults $w_h$=5, $w_{\text{link}}$=10, $\tau$=3, $c_{\text{pen}}$=15. Gate $g$=$(q_1, q_2)$ has $d_{\text{core}}(C_1, C_5)$=2 and is the only pending gate on $q_1$, so $\Delta_F$ is evaluated from $g$ alone and $\tilde{\Delta}_E$=0 (empty extended set). With $p_1$=$(0, 2) \in C_1$ and $p_2$=$(1, 1) \in C_5$, the shortest path on $G_{\mathcal{A}}$ gives $d_{\text{old}}^g$=28 (via $C_1 \to C_2 \to C_5$). Table I scores the three outgoing-link candidates: A wins via cheap staging combined with positive $\Delta_F$ and hop gain; B loses

because $q_1$ moves away from $C_5$ ($\Delta_F$ negative); C matches A on $\Delta_F$ and hop gain but is killed by the capacity penalty on the nearly-full landing core.

TABLE I
SCORE BREAKDOWN FOR THE THREE CANDIDATES OF THE RUNNING EXAMPLE (FIGURE 4). LOWER IS BETTER; CANDIDATE A IS SELECTED.

| Candidate | next core | direction | $d_{\text{prep}}$ | $c_{\text{cap}}$ | $g_{\text{hop}}$ | $\Delta_F$ | score $s$ |
|---|---|---|---|---|---|---|---|
| A | $C_2$ | towards $C_5$ | 1 | 0 | +5 | +12 | **−16** |
| B | $C_0$ | away from $C_5$ | 2 | 0 | −5 | −11 | 18 |
| C | $C_4$ | towards $C_5$ | 3 | 15 | +5 | +12 | 1 |

### D. Inter-Core Extended-Set Construction

The lookahead $\tilde{\Delta}_E$ in Eq. 4 depends on how the inter-core extended set $E$ is built. SABRE [13] fills $E$ in topological order, interleaving all wires uniformly; TELESABRE [15] uses BFS layers but within each layer emits gates in arbitrary order and weights each by its iteration index $g$ via $(1+g/10)$ rather than by depth. Both schemes can deprioritise the very follow-on gates a candidate teleport would help, causing $\tilde{\Delta}_E$ to undercount its benefit.

DSABRE keeps a BFS-layer expansion of the remaining DAG with two refinements: (i) within each layer, gates sharing a qubit with the front are emitted first; (ii) every gate in BFS layer $k$ is assigned $\text{dep}(g)=k$, used in the decay $\gamma^{\text{dep}(g)}$, so the size-$L$ truncation cuts cleanly at a layer boundary and weighting decays exponentially with depth rather than growing linearly with iteration index. It adds no hyperparameter and runs in $O(|E|_{\max})$ time per call when remaining in-degrees are maintained incrementally: BFS peels zero-in-degree gates layer by layer, each emitted gate triggering $O(1)$ decrements on its successors. The empirical contribution over the topological variant is reported in Section IV-C.

### E. Proactive Congestion Relief

In dense layouts, a core can become saturated: many pending gates demand qubits from it simultaneously and the core has no free slots to absorb incoming teleports. Standard reactive scoring (Eq. 4) does not address this until the bottleneck triggers a deadlock.

DSABRE adds a *proactive congestion relief* mechanism that fires alongside the gate-driven scoring of Eq. 4.

**Demand vector.** At each iteration the router computes a demand vector $d[c]$ over all cores by counting inter-core gates in the front and lookahead windows whose shortest core-graph path traverses $c$. $d[c]$ is therefore an upper bound on the number of upcoming teleports that will need to pass through $c$, regardless of whether $c$ itself hosts an operand.

**Trigger.** A core $c$ is flagged when both $d[c] \geq \theta_d$ (default $\theta_d$=3) and $f_c \leq \theta_f$ (default $\theta_f$=2) hold: high incoming demand combined with low free capacity. The reactive scorer alone would only respond once $f_c$ hits zero and the next gate-driven teleport fails to find a landing port; firing earlier on the conjunction avoids the eviction-cascade pattern that triggers deadlock.

**Relief candidates.** For each flagged core, the router generates teleports that move the most-idle logical qubits in $c$ — those whose next pending gate has the largest DAG depth from the front — into a less-loaded neighbour. Each candidate is scored by Eq. 4 with $\Delta_F$=0 — victims are restricted to qubits absent from the front layer, so no pending gate motivates the move and the hop-gain term is also omitted (no gate-defined target core). The decay-weighted lookahead $\tilde{\Delta}_E$ is retained, so that when a victim does have an upcoming gate within the extended-set window the landing port is preferred to be closer to its next partner. A *relief bonus* $-b_r\,(d[c]-f_c)$ proportional to the demand–capacity imbalance is added on top. Relief candidates compete directly against gate-driven candidates and win only when the bonus outweighs the staging cost; the mechanism therefore introduces no extra teleports when the front-layer scorer already has a cheap on-target move available. On the per-core adaptive layout the ablation in Section IV-C shows relief is close to gmean-neutral (disabling it *lowers* 64q gmean EPR by 8.7%): the layout now gives every core escape capacity, so the standing-congestion the relief mechanism was designed to pre-empt is largely absent, and its remaining contribution is confined to individual hard instances such as the 64q Random circuit.

### F. Checkpoint–Rollback Deadlock Escape

When Eq. 4 stalls, DSABRE falls back to LIGHTSABRE's release-valve idea [16]: force shortest-path progress on the most-stuck front-layer gate. We wrap it in a checkpoint: every iteration that reduces $|W|$ saves $(\ell, W)$, and after $L_{\text{deadlock}}$ stalled iterations the router restores the checkpoint before forcing intra-core SWAPs or hop-by-hop teleportation along $\text{core\_path}(c_1, c_2)$, aborting after $N^{\max}_{\text{backup}}$ failed recoveries. The escape is dormant on best-EPR runs and only salvages individual SabreLayout seeds on dense large circuits (e.g. one of three seeds on 200q QFT, with 17 activations).

### G. Computational Complexity

Let $N$=$|C|$, $n$ be the circuit width, $K$ the number of cores, $M$ qubits per core ($P$=$KM$), and $L$ the extended-set capacity (constant). Architecture distance tables are precomputed once in $O(KM^2 \log M + K^2 \log K)$ (Section II-C) and are not charged per circuit.

Per iteration, inter-core scoring generates $O(n)$ candidates each costing $O(L)$, and intra-core scoring evaluates $O(M)$ SWAP candidates per core at cost $O(|F_c| + L)$, summing to $T_{\text{iter}} = O(Mn + P)$. In the common case $n \geq K$ (e.g. $n/K \in \{6.25, 9.0, 10.7\}$ on our suites) this is $O(Mn)$. The extended set is rebuilt at most $N$ times at $O(KL)$ each, adding $O(NK)$ total. On a grid-of-grids the physical diameter is $O(\sqrt{P})$, bounding inserted operations by $O(N\sqrt{P})$; the checkpoint–rollback escape adds a constant $N^{\max}_{\text{backup}} \cdot L_{\text{deadlock}}$ iterations. Combining,

$$T_{\text{total}} = O\Big(N\sqrt{P} \cdot Mn\Big). \tag{5}$$

Compared to flat SABRE [13] on the same $P$ qubits ($O(Pn)$ per iteration), restricting intra-core scoring to core-local edges

TABLE II
BENCHMARK CIRCUIT PROPERTIES (MEASUREMENTS/BARRIERS STRIPPED)

| Suite | Circuit | Qubits | CX gates | Depth | CX/qubit |
|---|---|---|---|---|---|
| 25q | AE | 25 | 558 | 395 | 22.3 |
| | QFT | 25 | 580 | 173 | 23.2 |
| | QNN | 25 | 1223 | 259 | 48.9 |
| | Random | 25 | 1124 | 589 | 45.0 |
| | GHZ | 25 | 24 | 27 | 1.0 |
| | Graphstate | 25 | 25 | 19 | 1.0 |
| 36q | BV | 36 | 17 | 23 | 0.5 |
| | DJ | 36 | 35 | 41 | 1.0 |
| | W-state | 36 | 70 | 145 | 1.9 |
| | VQE-SU2 | 36 | 105 | 56 | 2.9 |
| | QPEexact | 36 | 1019 | 347 | 28.3 |
| | QAOA | 36 | 1200 | 256 | 33.3 |
| 64q | AE | 64 | 1962 | 1058 | 30.7 |
| | QFT | 64 | 1966 | 446 | 30.7 |
| | QNN | 64 | 8126 | 650 | 127.0 |
| | Random | 64 | 1627 | 403 | 25.4 |
| | GHZ | 64 | 63 | 66 | 1.0 |
| | Graphstate | 64 | 64 | 25 | 1.0 |

TABLE III
HARDWARE COST AND HEURISTIC PARAMETERS

| Parameter | Symbol | Value |
|---|---|---|
| SWAP cost | $c_{\text{swap}}$ | 3 |
| Teleport (EPR) cost | $c_{\text{tele}}$ | 10 |
| Capacity threshold | $\tau$ | 3 |
| Capacity penalty | $c_{\text{pen}}$ | 15 |
| Inter-core link weight | $w_{\text{link}}$ | 10 |
| Hop-gain weight | $w_h$ | 5 |
| Extended-set weight | $w_e$ | 0.25 |
| Extended-set capacity | $L$ | 20 |
| Lookahead decay | $\gamma$ | 0.9 |
| Deadlock limit | $L_{\text{deadlock}}$ | 50 |
| Max rollbacks | $N_{\text{backup}}^{\max}$ | 50 |

yields a factor-$K$ per-iteration improvement that grows with the number of cores. Empirically this scales well in practice: the largest circuit we route (360-qubit QFT with 13,300 CX gates on a 486-qubit H-grid) routes in 801 s of pure-Python wall time (Section IV-F).

## IV. EXPERIMENTAL EVALUATION

### A. Setup

**Benchmarks.** We use three circuit sets from the MQT Bench suite [17], all in the IBM native-gate set (Qiskit opt3 transpilation, measurements and barriers removed).

*Suite 1 (25-qubit):* quantum amplitude estimation (AE), quantum Fourier transform (QFT), quantum neural network (QNN), random circuit (Random), GHZ state preparation, and graph-state preparation.

*Suite 2 (36-qubit):* six circuits spanning a broader range of algorithmic families and CX densities: Bernstein–Vazirani (BV, linear chain), Deutsch–Jozsa (DJ, oracle), W-state preparation (cascaded star), VQE with SU(2) ansatz (layered variational), QAOA (dense all-to-all), and quantum phase estimation (QPEexact, hierarchical).

*Suite 3 (64-qubit):* the same six circuit families as Suite 1 scaled to 64 logical qubits (AE, QFT, QNN, Random, GHZ, Graphstate), run on the larger H-grid architecture.

Circuit properties are summarised in Table II.

**Architecture.** Suites 1 and 2 use a $2\times2$ B-grid: four $4\times4$ cores (64 physical qubits, 4 EPR links). Suite 3 uses a $2\times3$ H-grid: six $4\times4$ cores (96 physical qubits, 7 EPR links). See Figure 2 for the full qubit-level topology. The 25q suite on the B-grid and the 64q suite on the H-grid follow the benchmark setup of TELESABRE [15] (we add a 36q B-grid suite to broaden circuit diversity).

**Baselines.** We compare against two state-of-the-art distributed routers: (i) TELESABRE [15], a SABRE-style router that jointly optimises intra-core SWAPs, teledata (qubit moves), and telegate (remote CX execution) operations; and (ii) `pytket-dqc` [4], a Quantinuum library that distributes circuits via *gate teleportation* with multi-gate amortisation using KaHyPar hypergraph partitioning. TELESABRE is run using its compiled C binary with the `optimize_initial` flag enabled (built-in layout heuristic + 3-success sampling) on the same device; `pytket-dqc` uses its `HypergraphPartitioning` allocator (best of 5 random seeds; its hypergraph encoding requires a minor weight-0 patch to run on the KaHyPar $\geq$ 1.3.5 release we use). Internally, we also compare against DSABRE-Topo, an ablation that replaces the BFS-layer extended set with a topological-order one (Section IV-C).

*Caveat on e-bit counting.* All routers report in the same unit (one EPR pair = one e-bit; "EPR" and "e-bit" are used interchangeably), but what each unit *buys* differs. DSABRE uses *teledata*: one EPR per inter-core hop relocates a logical qubit, after which subsequent intra-core gates on it are free. `pytket-dqc` uses *gate teleportation*: one EPR distributed along a Steiner tree of cores executes a whole hyperedge of CNOTs sharing one control, paying one EPR per Steiner edge rather than per CNOT. Circuits with isolated inter-core gates favour teledata; circuits with long shared-control CNOT runs (QFT, QPE) favour gate teleportation. The $\Delta_{\text{tket}}$ columns in Tables IV–X are unit-consistent but should be read with this work-per-unit asymmetry in mind.

**Parameters.** Table III lists all hyperparameters. DSABRE is deterministic given a fixed layout; we run it with three SabreLayout seeds and report the best EPR count (lower = better). TELESABRE is stochastic and is run with three random seeds; again the best result is reported, following established practice [13]. Layout passes: 2 (forward–backward–forward).

*On best-of-$N$ reporting.* Each router uses its authors' convention: best-of-3 seeds for DSABRE and TELESABRE [15], best-of-5 for `pytket-dqc` [4]. This reflects the operational cost rather than per-attempt variance. The advantage over TELESABRE is robust to this choice: replacing best with median on 64q shifts the gmean gap from $-52.6\%$ to $-50.2\%$ vs. TELESABRE, still a large reduction. The comparison against `pytket-dqc` is method-dependent (Tables IV–VI) and is discussed per suite above.

TABLE IV
25-QUBIT SUITE (B-GRID 2×2 4×4, 64 PHYSICAL QUBITS). EPR = TELEDATA + TELEGATE (1 EPR PAIR EACH) FOR TELESABRE, TELEDATA FOR DSABRE, E-BITS FOR PYTKET-DQC; SWAP COUNTS INTRA-CORE SWAPS ONLY (PYTKET-DQC DOES NOT MODEL INTRA-CORE ROUTING). $\Delta_{\mathrm{TS}}$ AND $\Delta_{\mathrm{tket}}$ ARE DSABRE'S EPR CHANGE RELATIVE TO TELESABRE AND PYTKET-DQC RESPECTIVELY; NEGATIVE VALUES INDICATE A REDUCTION IN EPR COST BY DSABRE (LOWER IS BETTER). TELESABRE NUMBERS ARE BEST-OF-3-SEED; PYTKET-DQC USES ITS STRONGEST DISTRIBUTOR, COVEREMBEDDINGSTEINERDETACHED (STEINER-TREE GATE EMBEDDING WITH DETACHED GATES), BEST-OF-5-SEED; DSABRE RUNS WITH THE DEFAULT CONFIGURATION DESCRIBED ABOVE. COST ANALYSIS IS DEFERRED TO SECTION IV-G.

| | | TELESABRE | | DSABRE | | tket | ΔEPR (%) | |
|---|---|---|---|---|---|---|---|---|
| Circuit | CX | EPR | SWAP | EPR | SWAP | e-bits | TS | tket |
| ae | 558 | 23 | 297 | 23 | 284 | 48 | +0.0 | −52.1 |
| ghz | 24 | 2 | 29 | 1 | 6 | 3 | −50.0 | −66.7 |
| graphstate | 25 | 11 | 51 | 2 | 15 | 4 | −81.8 | −50.0 |
| qft | 580 | 39 | 355 | 33 | 334 | 61 | −15.4 | −45.9 |
| qnn | 1223 | 51 | 587 | 48 | 663 | 115 | −5.9 | −58.3 |
| random | 1124 | 292 | 1273 | 178 | 1241 | 479 | −39.0 | −62.8 |
| **gmean** | | 25.8 | 221 | 15.3 | 138 | 35.3 | **−40.6** | **−56.6** |

### B. Main Results

TELESABRE [15] supports three operation types: intra-core SWAPs, teledata (qubit movement, 1 EPR), and telegate (remote CX execution via the cat-entangler protocol, 1 EPR). We report TELESABRE's EPR count as the sum of teledata and telegate operations, so each EPR pair is counted exactly once. Unless stated otherwise, DSABRE is run with its default initial-mapping configuration: Qiskit `SabreLayout` on the corners-removed architecture graph, best of three seeds, followed by SABRE-style forward→backward→forward routing passes (Section IV-D). Subsequent tables and ablations inherit this default. We report two metrics in the main tables: EPR pairs and intra-core SWAPs. Cost analysis is deferred to Section IV-G.

**25q.** DSABRE reduces geometric-mean EPRs over TELESABRE by **40.6%** and over pytket-dqc by **56.6%** (Table IV). Per-circuit reductions versus TELESABRE range from parity on AE to −81.8% on Graphstate, with no regressions on this suite. Even against pytket-dqc's strongest Steiner-embedding distributor, DSABRE wins on all six circuits (−45.9 to −66.7%): state teleportation amortises a single EPR pair over many subsequent local gates, and at this scale that outweighs the gate-embedding savings pytket-dqc extracts by reusing shared control qubits.

**36q.** DSABRE reduces geometric-mean EPRs by **46.3%** vs. TELESABRE and **12.3%** vs. pytket-dqc (Table V). Per-circuit reductions versus TELESABRE are large (−35% to −80%, DJ at parity). The simple structured circuits BV, VQE-SU2, and W-state are essentially solved (1 EPR on BV; 6–10 EPRs on the others). Against pytket-dqc's Steiner-embedding distributor the picture is mixed: DSABRE matches or beats it on four circuits (BV −66.7%, QAOA −7.4%; DJ and W-state at parity), while the embedding is more e-bit-efficient on QPEexact (+32.7%) and VQE-SU2 (+11.1%), whose interaction graphs KaHyPar partitions almost perfectly; the geometric mean still favours DSABRE.

TABLE V
36-QUBIT SUITE (SAME B-GRID AS 25-QUBIT SUITE). METHODOLOGY AND NOTATION AS TABLE IV.

| | | TELESABRE | | DSABRE | | tket | ΔEPR (%) | |
|---|---|---|---|---|---|---|---|---|
| Circuit | CX | EPR | SWAP | EPR | SWAP | e-bits | TS | tket |
| bv | 17 | 5 | 34 | 1 | 12 | 3 | −80.0 | −66.7 |
| dj | 35 | 3 | 53 | 3 | 23 | 3 | +0.0 | +0.0 |
| qaoa | 1200 | 232 | 1027 | 137 | 1015 | 148 | −40.9 | −7.4 |
| qpeexact | 1019 | 100 | 771 | 65 | 586 | 49 | −35.0 | +32.7 |
| vqe_su2 | 105 | 16 | 80 | 10 | 56 | 9 | −37.5 | +11.1 |
| wstate | 70 | 12 | 88 | 6 | 25 | 6 | −50.0 | +0.0 |
| **gmean** | | 20.1 | 147 | 10.8 | 78 | 12.3 | **−46.3** | **−12.3** |

**64q.** The H-grid suite is where the gap from TELESABRE is most consequential (Table VI). DSABRE reduces gmean EPRs by **52.6%** over the five circuits on which TELESABRE converges, with per-circuit reductions ranging from −18.2% (GHZ) to −78.9% (Graphstate) and QFT/QNN at −50.2%/−62.8%. The per-core adaptive corner reservation (Section **??**) supplies every core with escape slots, which resolves the earlier star-circuit weakness on GHZ: it now improves (9 vs. 11) rather than regressing, as no core is born fully packed and the local Δ-form score is no longer forced to chase the next CX out of a saturated core. Against pytket-dqc's strongest distributor the comparison *reverses*: the 6-circuit geometric mean is +21.3%, i.e. DSABRE's teleport-EPRs exceed pytket-dqc's e-bits. COVEREMBEDDINGSTEINERDETACHED is markedly more e-bit-efficient on the structured circuits (AE +71.6%, GHZ +80.0%, Graphstate +77.8%, QFT +12.7%), where Steiner-tree gate embedding reuses a few EPR pairs across many controlled operations; DSABRE leads only on the two densest circuits (QNN −30.9%, Random −25.5%), on which the embedding method does not scale or embed and a plain partitioner is used. Two caveats temper this: pytket-dqc's e-bit count omits the intra-core SWAP cost that DSABRE pays and reports, and its strongest method is computationally far heavier (minutes to intractable per large circuit versus DSABRE's seconds). On Random, TELESABRE fails to converge on any of three seeds; DSABRE completes (721 EPR) under default scoring and proactive relief alone, without triggering checkpoint–rollback.

*a) Shared-mapping comparison.:* To isolate routing from allocation quality, we re-run DSABRE from each baseline's own initial layout. The TELESABRE gap persists (−25% EPR on 25q, −43% on 64q from its `optimize_initial` layout); against `pytket-dqc`'s KaHyPar partition DSABRE wins on all three suites (−55% on 25q, −14% on 36q, −36% on 64q); the 36q margin is the narrowest because the sparse interaction graphs in that suite are already near-optimally partitioned by KaHyPar.

### C. Ablation Study

We conduct a systematic ablation of DSABRE's design choices across three axes: (i) the extended-set construction, (ii) discrete mechanism on/off, and (iii) continuous parameter sensitivity. All ablations inherit DSABRE's default configuration (Section IV-B) and report the *geometric-mean EPR count*

TABLE VI
64-QUBIT SUITE (H-GRID 2×3 4×4, 96 PHYSICAL QUBITS). METHODOLOGY AND NOTATION AS TABLE IV; *TELESABRE FAILS TO CONVERGE ON RANDOM. ‡PYTKET-DQC'S COVEREMBEDDINGSTEINERDETACHED DOES NOT SCALE TO (QNN) OR EMBED (RANDOM) THESE DENSE CIRCUITS WITHIN A 20-MINUTE BUDGET; THE NON-EMBEDDING PARTITIONINGHETEROGENEOUS DISTRIBUTOR IS USED FOR THOSE TWO CELLS.

| | | TELESABRE | | DSABRE | | tket | ΔEPR (%) | |
|---|---|---|---|---|---|---|---|---|
| Circuit | CX | EPR | SWAP | EPR | SWAP | e-bits | TS | tket |
| ae | 1962 | 323 | 1508 | 242 | 1493 | 141 | −25.1 | +71.6 |
| ghz | 63 | 11 | 111 | 9 | 57 | 5 | −18.2 | +80.0 |
| graphstate | 64 | 76 | 379 | 16 | 89 | 9 | −78.9 | +77.8 |
| qft | 1966 | 410 | 1872 | 204 | 1563 | 181 | −50.2 | +12.7 |
| qnn | 8126 | 1365 | 6292 | 508 | 5229 | 735‡ | −62.8 | −30.9 |
| random* | 1627 | — | — | 721 | 3608 | 968‡ | — | −25.5 |
| **gmean** (5) | | 172.1 | 943 | 81.6 | 573 | 61.0 | **−52.6** | **+33.7** |
| **gmean** (6) | | — | — | 117.3 | 779 | 96.7 | — | **+21.3** |

TABLE VII
ABLATION OF THE INTER-CORE EXTENDED-SET CONSTRUCTION. BOTH VARIANTS ARE THE SAME ROUTER DIFFERING ONLY IN HOW THE INTER-CORE $E$ IS BUILT: TOPOLOGICAL ORDER VS. BFS-LAYER EXPANSION (DEFAULT). GEOMETRIC MEAN OVER EACH SUITE.

| | DSABRE-topo | | DSABRE | | Δ (%) | |
|---|---|---|---|---|---|---|
| Suite | EPR | SWAP | EPR | SWAP | EPR | SWAP |
| **25q** | 16.4 | 141 | 15.3 | 138 | −6.8 | −2.1 |
| **36q** | 12.1 | 94 | 10.8 | 78 | −10.9 | −16.5 |
| **64q** | 127.4 | 832 | 117.3 | 779 | −7.9 | −6.3 |

(gmEPR) over the respective suite, so the DSABRE / **Full** baselines in Tables VII and VIII match the headline gmEPR in Tables IV/VI.

*a) Inter-core extended-set construction: BFS vs. topological.:* To isolate the contribution of the BFS-layer inter-core extended set (Section III-D) we substitute the standard topological-order construction in the teleport scorer (the intra-core extended set is unchanged) and rerun all three suites. Table VII reports the result.

BFS reduces gmean EPRs by 6.8%, 10.9%, and 7.9% on the three suites respectively. The gain is largest on the 36q suite and reflects the richer dependency structure in larger circuits, where topological order drifts further from the active front layer. SWAP counts fall on all three suites (−2.1%, −16.5%, −6.3%): the BFS scorer resolves the front through better-placed teleportations and leaves less residual intra-core work.

*b) Mechanism ablation.:* Table VIII ablates each scoring component in isolation on the 25q and 64q suites by setting the corresponding parameter to zero or disabling the mechanism entirely. The two suites stress different regimes: the 25q B-grid is sparsely loaded (qubits per core $\leq 7$, ample headroom on every core), so congestion-driven mechanisms barely activate, while the 64q H-grid runs at ∼67% qubit utilisation and saturates inter-core ports under dense workloads.

The capacity penalty is the single most critical component on both suites: without it the router greedily teleports into full cores, triggering cascading evictions that inflate gmean EPR by $\sim 61\%$ on 25q and $\sim 54\%$ on 64q. The extended-

TABLE VIII
MECHANISM ABLATION ON THE 25Q AND 64Q SUITES. ONE COMPONENT IS DISABLED PER ROW; ALL OTHERS REMAIN AT THEIR DEFAULTS. SAME PROTOCOL AS THE MAIN RESULTS, SO THE **FULL** BASELINE MATCHES THE HEADLINE GMEPR IN TABLES IV AND VI.

| | 25q | | 64q | |
|---|---|---|---|---|
| Configuration | gmEPR | Δ (%) | gmEPR | Δ (%) |
| **Full** (baseline) | 15.3 | — | 117.3 | — |
| No extended-set lookahead ($w_e = 0$) | 17.1 | +11.3 | 148.3 | +26.5 |
| No capacity penalty ($c_{\text{pen}} = 0$) | 24.7 | +60.8 | 180.0 | +53.5 |
| No hop-gain reward ($w_h = 0$) | 15.3 | 0.0 | 119.5 | +1.8 |
| No congestion relief | 15.3 | 0.0 | 107.1 | −8.7 |

set lookahead contributes +11% on 25q and +27% on 64q; removing it collapses the scorer to a near-myopic policy, and its value grows with circuit size as dependency chains lengthen. The hop-gain reward — a directional bonus for moves that reduce inter-core hop count — is inert on the 25q B-grid (where the front-layer weighted distance already captures direction) but contributes a small improvement on the larger 64q H-grid (+1.8% gmean): longer inter-core paths mean the specific landing port can be far from the optimal staging position for the next hop, depressing $\Delta_F$ below the true directional value; $g_{\text{hop}}$ compensates with a fixed position-independent signal.

Congestion relief is near-neutral on both suites once the per-core adaptive layout is in place: it is inactive on 25q (0%) and mildly counterproductive on 64q (−8.7%, i.e. disabling it lowers gmean EPR). Because every core now begins with reserved escape capacity, the standing congestion the relief mechanism was designed to pre-empt is largely absorbed upstream by the layout; its residual value is confined to individual hard instances (e.g. 64q Random, where it enables completion) rather than the suite geometric mean.

*c) Parameter sensitivity.:* We sweep each continuous hyperparameter one-at-a-time on 25q and 64q. All defaults—$w_e$=0.25, $c_{\text{pen}}$=15, $|E|$=20, $\gamma$=0.9, $w_{\text{link}}$=10, and the relief bonus—sit on a flat region of their respective curves (within $\sim$ 2% of the local minimum), so performance is robust to small perturbations. The only sharp regressions occur when a control is pushed far from its default: e.g. $w_e \geq 1.0$ inflates 25q gmEPR by 3−5× as the lookahead dominates immediate preparation costs, $c_{\text{pen}}$=0 doubles gmEPR by inviting deadlocks in saturated cores, and $\gamma$=1.0 (flat weighting) costs 7% on 64q. These observations confirm that the defaults are well-calibrated and that no single hyperparameter is a hidden lever; tuning beyond the defaults yields diminishing returns.

### D. SabreLayout-Based Initial Layout

DSABRE bootstraps from Qiskit `SabreLayout` [13] run on the full architecture graph $G_{\mathcal{A}}$ (intra-core edges plus inter-core links), with two adaptations for the distributed setting.

*Corner removal.* SabreLayout packs each core to its full physical capacity. Two problems follow. First, an occupied communication port cannot accept an incoming teleport without first evicting its current occupant, so saturated cores either stall the inter-core scorer or force chains of relocations.

TABLE IX
COMPILATION TIME IN SECONDS (64-QUBIT SUITE, H-GRID 2×3 4×4). TELESABRE: BEST-SEED WALL-CLOCK TIME. DSABRE: TOTAL TIME FOR THREE ROUTING PASSES (FWD→BWD→FWD) FROM THE BEST SABRELAYOUT SEED. TELESABRE IS A COMPILED C++ BINARY; DSABRE IS PURE-PYTHON WITH NETWORKX. *TELESABRE FAILS TO CONVERGE ON RANDOM.

| Circuit | CX | TELESABRE (s) | DSABRE (s) |
|---|---|---|---|
| AE | 1962 | 1.05 | 12.60 |
| GHZ | 63 | 0.02 | 0.09 |
| Graphstate | 64 | 3.92 | 0.05 |
| QFT | 1966 | 1.81 | 10.34 |
| QNN | 8126 | 25.21 | 123.60 |
| Random* | 1627 | — | 16.69 |

Second, full cores trip the capacity penalty (Section III-C) on every candidate teleport into them, biasing the router away from the actually-preferred landing core. We exclude the four lowest-degree *corner* physical qubits from the coupling map passed to SabreLayout—in both B-grid and H-grid architectures they are furthest from any inter-core link—which guarantees four free slots per affected core to serve as eviction scratch space and to keep the core below the capacity threshold.

*Forward–backward–forward schedule.* We run three routing passes (forward → reversed-DAG backward → forward) and keep the better of the two forward passes by EPR. The backward pass starts from the first forward pass's output layout $\ell_f$ and routes the reversed DAG, producing $\ell_b$ that pre-positions qubits near their early interaction partners; the third pass then warm-starts from $\ell_b$. This is the same mechanism as SABRE's backward refinement [13] and outperforms pure-forward warm-restart. Best of three SabreLayout seeds is reported.

### E. Compile Time

Table IX reports wall-clock compilation time on the 64-qubit suite.

Times scale with CX count: TELESABRE spans 0.02 s (GHZ) to 25 s (QNN, 8126 CX); DSABRE spans 0.05 s to 124 s. DSABRE is 5–12× slower on circuits where both converge, which we attribute to the pure-Python vs. compiled-C++ constant factor and to per-iteration NetworkX shortest-path lookups that TELESABRE caches. The Graphstate row inverts this ordering (0.05 s vs. 3.92 s) because TELESABRE's 3-success sampling spends many iterations before three runs converge on that circuit.

### F. Large-Circuit Scalability

To stress-test DSABRE substantially beyond the main evaluation range we run DSABRE, TELESABRE, and `pytket-dqc` on three additional suites using the MQT-Bench QFT circuit (IBM native-gate, Qiskit opt3). The architectures scale the H-grid of the 64q suite:

- **100q**: H-grid 2×3 5×5 (150 physical).
- **200q**: H-grid 4×3 5×5 (300 physical).
- **360q**: H-grid 2×3 9×9 (486 physical).

TABLE X
LARGE-CIRCUIT SCALABILITY (QFT). EACH ROUTER RUNS FROM ITS OWN INITIAL LAYOUT. NEGATIVE ΔEPR IS BETTER. TELESABRE FAILS TO CONVERGE ON THE 200Q CIRCUIT WITHIN THE 600 S TIMEOUT. †SEE CAVEAT ON E-BIT COUNTING. ‡`PYTKET-DQC`'S COVEREMBEDDINGSTEINERDETACHED EXCEEDS A 25-MINUTE BUDGET AT 200Q; THE PARTITIONINGHETEROGENEOUS DISTRIBUTOR IS USED. §360Q IS NOT YET REGENERATED ON THE PER-CORE ADAPTIVE LAYOUT; ITS DSABRE COLUMN AND `PYTKET-DQC` E-BITS ARE FROM THE PREVIOUS CONFIGURATION (HYPERGRAPHPARTITIONING).

| | | TELESABRE | | DSABRE | | tket† | ΔEPR (%) | |
|---|---|---|---|---|---|---|---|---|
| Suite | CX | EPR | SWAP | EPR | SWAP | e-bits | TS | tket |
| 100q | 3420 | 248 | 2414 | 223 | 2723 | 114 | −10.1 | +95.6 |
| 200q | 7220 | — | — | 834 | 7043 | 447‡ | — | +86.6 |
| 360q§ | 13300 | 1109 | 15098 | 579 | 27489 | 669 | −47.8 | −13.5 |

TELESABRE is best-of-3-seed with a 600 s per-seed timeout for 200q+; DSABRE uses its default configuration; `pytket-dqc` uses HypergraphPartitioning (best of 5 seeds). Each router runs from its own initial layout, so the $\Delta_{\text{tket}}$ column should be read with this layout asymmetry in mind in addition to the e-bit counting caveat (Section IV-A).[1]

Table X shows the QFT scalability trend. At *100q*, DSABRE now edges TELESABRE (−10.1% EPR under the per-core adaptive layout), but pytket-dqc's Steiner-tree embedding is far more e-bit-efficient on this deep controlled-phase chain (+95.6%: DSABRE uses 223 teleport-EPRs versus 114 e-bits). At *200q*, TELESABRE fails to converge within the 600 s timeout, whereas DSABRE completes (834 EPR) under its default scoring and proactive relief; the checkpoint–rollback escape (Section III-F) does not fire on the best-EPR run, but is what salvages one of the three SabreLayout seeds that would otherwise abort, leaving the best-of-three selection well-fed. The embedding advantage persists here (+86.6% vs. `pytket-dqc`), though COVEREMBEDDINGSTEINERDETACHED exceeds a 25-minute budget and the cheaper PARTITIONINGHETEROGENEOUS distributor is reported instead. The *360q* row is carried over from the previous configuration pending regeneration on the per-core layout (Section **??**). As at the smaller sizes, the QFT comparison against pytket-dqc is dominated by its gate-teleportation embedding on this circuit family, which DSABRE's state-teleportation model does not target; the EPR-versus-e-bit caveat (no intra-core SWAP cost for pytket-dqc) applies throughout. Compile times for 360q QFT are 281 s (TELESABRE, C++, best of 3 seeds), 801 s (DSABRE, Python, SabreLayout + fwd→bwd→fwd), and 546 s (`pytket-dqc`, KaHyPar, best of 5 seeds); the Python overhead is the primary driver of DSABRE's longer wall time relative to the compiled C++ TELESABRE.

[1]For a layout-controlled point of comparison, re-running DSABRE from `pytket-dqc`'s HypergraphPartitioning layout gives EPR counts of 315 (100q), 880 (200q), and 385 (360q), versus `pytket-dqc`'s 815, 1124, and 669 — corresponding to $\Delta_{\text{tket}}$ of −61.4%, −21.7%, and −42.5%. In this matched-layout setting DSABRE beats `pytket-dqc` at all three sizes, including the 200q row where the SabreLayout configuration is marginally worse.

TABLE XI
COST-MODEL SENSITIVITY: GEOMETRIC-MEAN COST REDUCTION OF DSABRE OVER TELESABRE AS $c_{\text{tele}}$ VARIES ($c_{\text{swap}} = 3$ FIXED). NEGATIVE IS BETTER. 64Q EXCLUDES THE RANDOM CIRCUIT ON WHICH TELESABRE ABORTS.

| $c_{\text{tele}}$ | 25q | 36q | 64q |
|---|---|---|---|
| 10 | **−39.2** | **−47.0** | **−44.6** |
| 20 | **−39.8** | **−47.0** | **−46.9** |
| 50 | **−40.3** | **−47.0** | **−49.5** |
| 100 | **−40.5** | **−46.8** | **−50.9** |

### G. Cost Analysis

The combined-cost ratio $c_{\text{tele}}$:$c_{\text{swap}}$ is hardware-dependent: near-term microwave-coupled modular superconducting qubits keep teleportation within a small constant factor of local SWAPs [1], while photonic-link architectures have measured EPR-generation times three to four orders of magnitude slower than local gates [9], [10], pushing the ratio towards 100:1 or beyond. Table XI sweeps $c_{\text{tele}}$ from 10 to 100 with $c_{\text{swap}} = 3$ held fixed, reporting geometric-mean cost reduction of DSABRE vs. TELESABRE.

The pattern is consistent across all three suites: the gap to TELESABRE widens as $c_{\text{tele}}$ grows. At the teleport-friendly end ($c_{\text{tele}}$=10, ratio $c_{\text{tele}}/c_{\text{swap}} \approx 3.3$) the cost model penalises teleports least, so TELESABRE's higher EPR count is discounted; DSABRE still saves 20.6% on 25q, 39.0% on 36q, and 23.8% on 64q. At $c_{\text{tele}}$=100 (ratio $\approx 33$) — the SWAP-favourable regime of near-term photonic-link hardware [9], [10] — DSABRE's smaller EPR count dominates and the savings reach 35.2%, 43.2%, and 38.9% respectively. Across the entire practically relevant range DSABRE dominates TELESABRE, and the advantage grows with EPR cost — exactly the regime where the cost model becomes stricter.

## V. RELATED WORK

The Introduction already places DSABRE within the partition–communicate–schedule decomposition of the DQC compilation stack and lists the major lines of work in each stage. Here we focus on what specifically distinguishes DSABRE from the closest prior art and on the single-QPU SABRE family that we build on but do not subsume.

**SABRE-style distributed routers.** TELESABRE [15] is our closest prior work and direct experimental baseline. Both routers follow the SABRE loop, but differ in how they score inter-core moves: TELESABRE evaluates candidates via a *contracted communication-graph energy* that averages per-gate Dijkstra distances over the whole front layer (extended in topological order), whereas DSABRE uses a local, gate-centric five-term score (Eq. 4) that explicitly accounts for intra-core staging cost, core capacity, and directional progress, paired with a BFS-layer extended set (Section III-D). This makes DSABRE's inter-core scoring directly comparable in form to its intra-core SWAP scoring (Eq. 3). DSABRE additionally provides proactive congestion-relief candidates (Section III-E) and a checkpoint–rollback deadlock escape (Section III-F), whereas TELESABRE relies on a safety-valve iteration cap; TELESABRE supports telegate candidates while DSABRE currently uses teledata only.

DMapS [14] is a concurrent end-to-end DQC compiler that pairs a KaHyPar partitioner with a SABRE-derived router and shares our joint EPR-plus-local-SWAP objective. Two design choices set it apart from DSABRE: its only cross-core primitive is a two-EPR *remote SWAP* (whereas DSABRE uses one-EPR one-way teledata into a layout-reserved free slot), and its initial allocation comes from a KaHyPar partition of the interaction hypergraph (whereas DSABRE uses SabreLayout on $G_{\mathcal{A}}$). On the dense circuits in our suites DSABRE consumes substantially fewer EPRs — e.g. 2−4× on 25q QFT/QNN/Random and 1.6−1.8× on 64q QFT/QNN/Random — while DMapS wins on maximally sparse circuits (GHZ, Graphstate, W-state) where KaHyPar partitions interaction graphs almost perfectly. A code-level audit, the full per-circuit head-to-head, and a controlled fill-ratio sweep showing parity once the device fill ratio approaches 1 are included in the online appendices accompanying the paper (see "Artefact availability" below).

Allocation methods built on hypergraph partitioning [3], [4] are complementary to DSABRE: any can supply the initial layout its routing loop consumes. Similarly, time-aware schedulers [11], [12] sit upstream or downstream of the routing stage DSABRE addresses.

**Single-QPU qubit mapping.** On a single-core processor the routing problem reduces to inserting intra-chip SWAPs. The closest single-QPU antecedent to our work is the front+extended-set scorer SABRE [13] that we and TELESABRE both build on. Babu et al. [18] use auxiliary-qubit gate teleportation on a heavy-hex IBM Eagle processor, achieving 10–25% depth reduction versus standard SABRE; the setting is single-QPU and not directly comparable to multi-core routing.

## VI. FUTURE WORK

**Composing DSABRE with burst execution.** DSABRE optimises EPR count on a fixed circuit DAG. Burst-execution passes such as AutoComm [8] and its collective extension CollComm [19] operate earlier in the toolchain: they use gate commutativity to reorder consecutive non-local gates on the same wire (or across multiple wires) and amortise the whole block onto a single shared entanglement state, reporting EPR savings of 50–75% on amenable circuits. The two approaches operate at different layers and should compose well: a burst-extraction pre-pass would expose longer wire-aligned gate sequences, and DSABRE's BFS extended set would let the teleport score capture their full benefit. Quantifying the joint improvement—especially on circuits where a burst's beneficiary core is itself a function of the routing decisions DSABRE makes—is an open question, and recent integrations of gate-teleportation amortisation with SABRE-style mappers [18] suggest the interaction is non-trivial.

**Parallel teleportation for circuit-runtime minimisation.** DSABRE optimises EPR count and emits one teleport per iteration; wall-clock runtime depends additionally on whether teleports with disjoint links and staging sets can fire concurrently. A natural follow-on is a scheduling pass over

DSABRE's teleport trace that batches non-conflicting moves, co-designed with denser inter-core links [20] and entanglement buffer qubits [19], [21]. Time-sliced compilers [11], [12] provide a starting point but are not paired with a SABRE-style scoring loop; naively adding links without retuning the routing energy can *increase* EPR consumption.

## VII. Conclusion

We presented DSABRE, a SABRE-style router for multi-core distributed quantum computers that, on each iteration of a lookahead-driven loop, resolves intra-core front-layer gates by SWAP scoring and only scores inter-core teleport candidates when the intra-core front is empty, with both phases sharing a common decay-weighted lookahead form. The empirical takeaway is that three design choices — an explicit capacity penalty that keeps the scorer from teleporting into saturated cores, proactive congestion relief triggered by an explicit demand vector, and a BFS-layer inter-core extended set that respects DAG dependencies — account for most of the gap to the state of the art, both individually and together, and that this gap widens as circuits and architectures scale. More broadly, the results indicate that distributed routing benefits less from a richer per-candidate scoring formula than from ensuring that the lookahead signal and the architectural capacity constraints enter the same scoring step.

*Artefact availability.:* The DSABRE implementation, the routed-circuit benchmark suites, per-circuit JSON results, and the online appendices referenced throughout this paper are available at https://github.com/ebony72/dsabre.

## Acknowledgments

This work was partially supported by the Australian Research Council (Grant No. DP250102952 and DP220102059). The author used Anthropic's Claude (via Claude Code) to assist with: (i) refactoring portions of the DSABRE implementation, (ii) authoring benchmark scripts and aggregating per-circuit JSON results into the tables of Section IV, (iii) drafting the LaTeX sources for the architecture and teleport-candidate figures (Figures 2 and 4), and (iv) copy-editing prose throughout the manuscript. All AI-generated content was reviewed, verified, and corrected by the author, who takes full responsibility for the correctness and originality of the work.

## References

[1] D. Cuomo, M. Caleffi, and A. S. Cacciapuoti, "Towards a distributed quantum computing ecosystem," *IET Quantum Communication*, vol. 1, no. 1, pp. 3–8, 2020.

[2] P. Escofet, A. Ovide, M. Bandic, L. Prielinger, C. G. Almudever, S. Feld, E. Alarcón, and S. Abadal, "Revisiting the mapping of quantum circuits: Entering the multi-core era," *ACM Transactions on Quantum Computing*, 2024, arXiv:2403.17205.

[3] P. Andres-Martinez and C. Heunen, "Automated distribution of quantum circuits via hypergraph partitioning," *Physical Review A*, vol. 100, no. 3, p. 032308, 2019.

[4] P. Andres-Martinez, D. Mills, T. Forrer, and L. Henaut, "CQCL/pytket-dqc," https://github.com/CQCL/pytket-dqc, Jun. 2024.

[5] C. H. Bennett, G. Brassard, C. Crépeau, R. Jozsa, A. Peres, and W. K. Wootters, "Teleporting an unknown quantum state via dual classical and Einstein-Podolsky-Rosen channels," *Physical Review Letters*, vol. 70, no. 13, pp. 1895–1899, 1993.

[6] J. Eisert, K. Jacobs, P. Papadopoulos, and M. B. Plenio, "Optimal local implementation of nonlocal quantum gates," *Physical Review A*, vol. 62, no. 5, p. 052317, 2000.

[7] A. Yimsiriwattana and S. J. Lomonaco Jr., "Generalized GHZ states and distributed quantum computing," *arXiv preprint quant-ph/0402148*, 2004.

[8] A. Wu, H. Zhang, G. Li, A. Shabani, Y. Xie, and Y. Ding, "AutoComm: A framework for enabling efficient communication in distributed quantum programs," in *Proceedings of the 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2022, arXiv:2207.11674.

[9] L. J. Stephenson, D. P. Nadlinger, B. C. Nichol, S. An, P. Drmota, T. G. Ballance, K. Thirumalai, J. F. Goodwin, D. M. Lucas, and C. J. Ballance, "High-rate, high-fidelity entanglement of qubits across an elementary quantum network," *Physical Review Letters*, vol. 124, no. 11, p. 110501, 2020.

[10] S. Daiss, S. Langenfeld, S. Welte, E. Distante, P. Thomas, L. Hartung, O. Morin, and G. Rempe, "A quantum-logic gate between distant quantum-network modules," *Science*, vol. 371, no. 6529, pp. 614–617, 2021.

[11] D. Ferrari, A. S. Cacciapuoti, M. Amoretti, and M. Caleffi, "Compiler design for distributed quantum computing," *IEEE Transactions on Quantum Engineering*, vol. 2, pp. 1–20, 2021.

[12] J. M. Baker, C. Duckering, A. Hoover, and F. T. Chong, "Time-sliced quantum circuit partitioning for modular architectures," in *Proceedings of the 17th ACM International Conference on Computing Frontiers (CF)*, 2020, pp. 98–107.

[13] G. Li, Y. Ding, and Y. Xie, "Tackling the qubit mapping problem for NISQ-era quantum devices," in *Proceedings of the 24th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2019, pp. 1001–1014.

[14] T. Luo, Y. Zheng, Y. Deng, and X. Fu, "DMapS: End-to-end qubit mapping and routing for distributed quantum computing architectures," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 2025, code: https://github.com/RoccoLoter/DMapS.

[15] E. Russo, E. Vinciguerra, M. Palesi, D. Patti, G. Ascia, and V. Catania, "TeleSABRE: Heuristic layout synthesis in multi-core quantum systems with teleport interconnect," in *Proceedings of the IEEE International Conference on Quantum Computing and Engineering (QCE)*, 2025, pp. 749–758, arXiv:2505.08928. Code: https://github.com/Haimrich/telesabre.

[16] H. Zou, M. Treinish, K. Hartman, A. Ivrii, and J. Lishman, "LightSABRE: A lightweight and enhanced SABRE algorithm," 2024. [Online]. Available: https://arxiv.org/abs/2409.08368

[17] N. Quetschlich, L. Burgholzer, and R. Wille, "MQT Bench: Benchmarking software and design automation tools for quantum computing," *Quantum*, vol. 7, p. 1062, 2023.

[18] A. P. Babu, O. Kerppo, A. Muñoz Moller, M. Haghparast, and M. Silveri, "Gate teleportation-assisted routing for quantum algorithms," 2025.

[19] A. Wu, Y. Ding, and A. Li, "CollComm: Enabling efficient collective quantum communication based on EPR buffering," 2022.

[20] P. Escofet, S. B. Rached, S. Rodrigo, C. G. Almudever, E. Alarcón, and S. Abadal, "Interconnect fabrics for multi-core quantum processors: A context analysis," in *Proceedings of the 16th International Workshop on Network on Chip Architectures (NoCArc)*, 2023, arXiv:2309.07313.

[21] J. Liu, A. Zang, M. Suchara, T. Zhong, and P. D. Hovland, "Hardware-software co-design for distributed quantum computing," in *2025 62nd ACM/IEEE Design Automation Conference (DAC)*, 2025, pp. 1–6.

**Sanjiang Li** received his B.Sc. in mathematics from Shaanxi Normal University in 1996 and his Ph.D. in mathematics from Sichuan University in 2001. He is currently a professor at the Centre for Quantum Software and Information at the University of Technology Sydney (UTS). Prior to joining UTS, he worked in the Department of Computer Science and Technology at Tsinghua University from 2001 to 2008. His primary research interests include knowledge representation, artificial intelligence, and quantum circuit compilation.